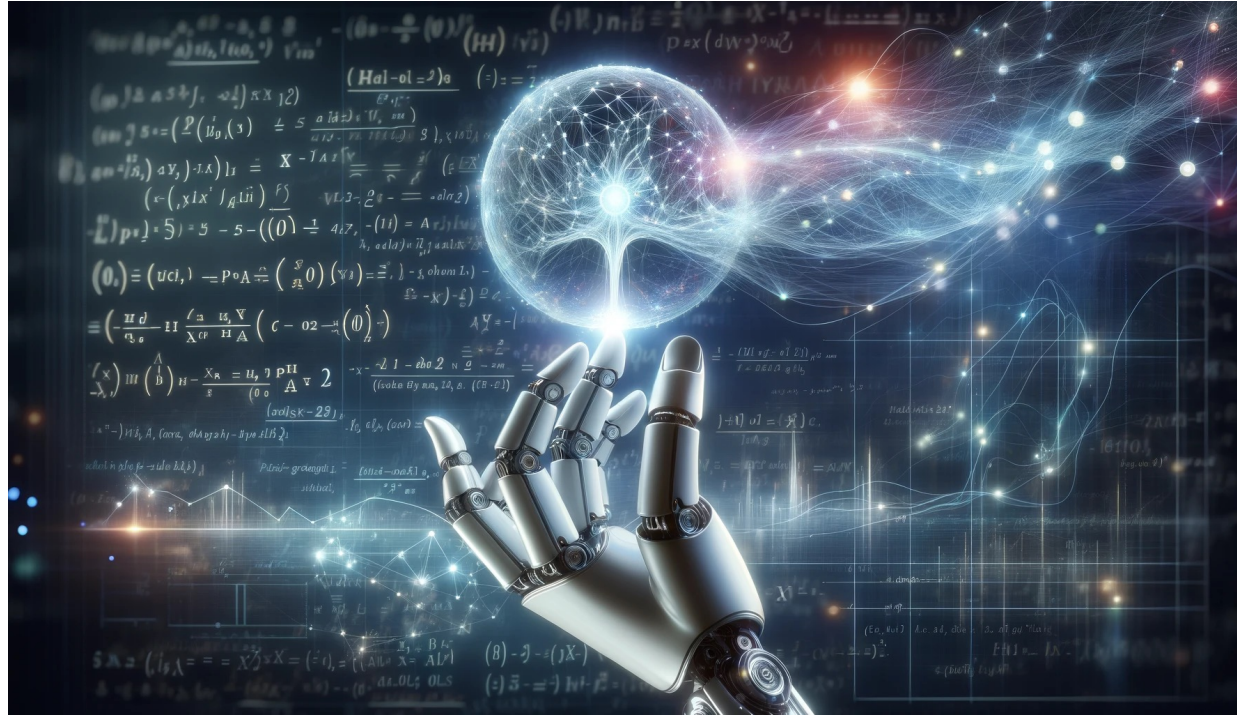


16-831: Introduction to Robot Learning, Spring 2024

Policy Gradient Methods



Generated by GPT-4 with DALLÉ

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831-s24.github.io/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☒ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning (one guest lecture)
- ☐ Topic group 6: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 7: Bandit, preference-based learning, exploration
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation & poster session

Outline

(Some black board derivation today!)

- ❑ Recap: Q-Learning
- ❑ Why and what is policy gradient
- ❑ Policy gradient theorem and REINFORCE
- ❑ Understanding and improving REINFORCE

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

(Some black board derivation today!)

☐ Recap: Q-Learning

☐ Why and what is policy gradient

☐ Policy gradient theorem and REINFORCE

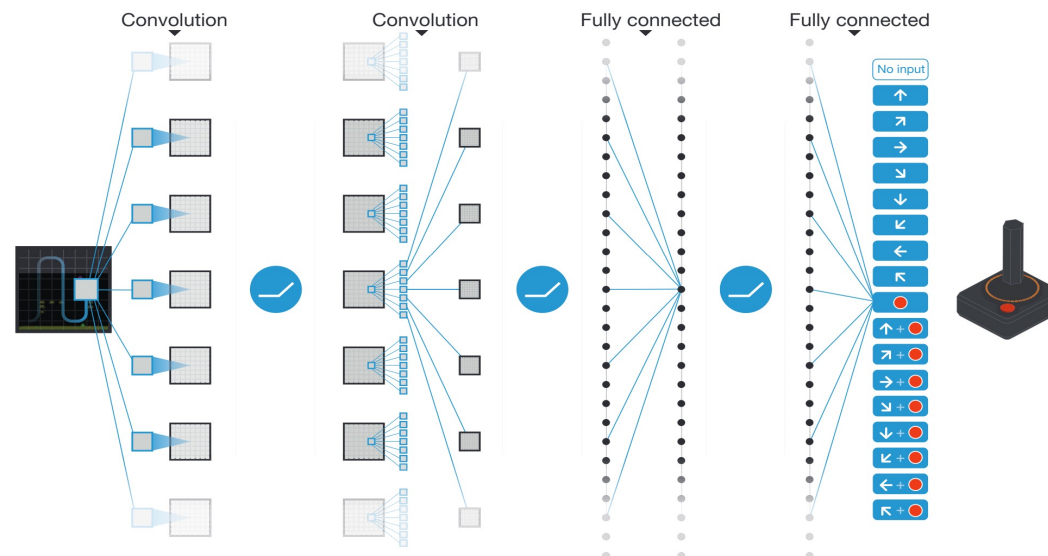
☐ Understanding and improving REINFORCE

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Recap: Deep Q-Learning

DQN (Deep Q-Network):

- ❑ Epsilon greedy policy gives a and observe (s, a, s', r)
- ❑ Update the replay buffer
- ❑ Sample a batch from the buffer and compute the target $y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_{\phi^-}(s'_i, a'_i)$
- ❑ Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$
- ❑ Periodically update ϕ^- : $\phi^- \leftarrow \phi$



$$\begin{bmatrix} Q_{\theta}(\mathbf{s}, \mathbf{a}^{(1)}) \\ Q_{\theta}(\mathbf{s}, \mathbf{a}^{(2)}) \\ \vdots \\ Q_{\theta}(\mathbf{s}, \mathbf{a}^{(k)}) \end{bmatrix}$$

Recap: Deep Q-Learning

DQN (Deep Q-Network):

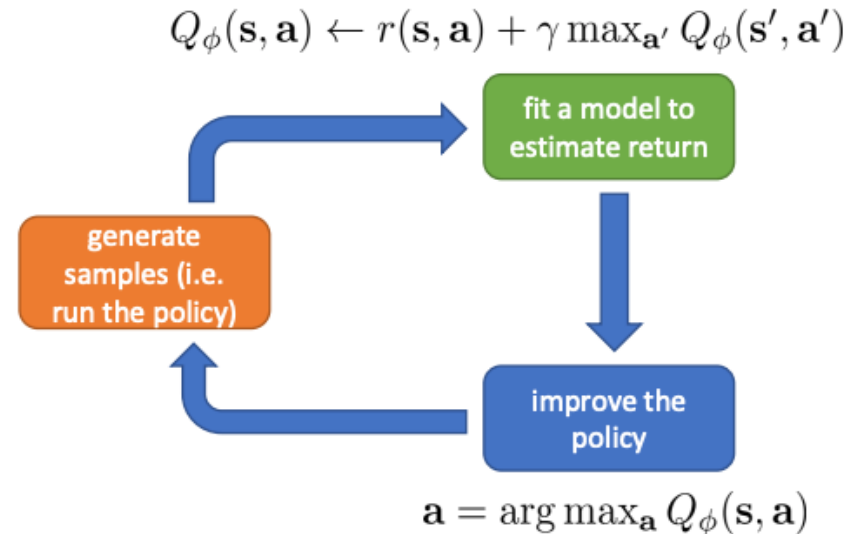
- ❑ Epsilon greedy policy gives a and observe (s, a, s', r)
- ❑ Update the replay buffer
- ❑ Sample a batch from the buffer and compute the target $y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_{\phi^-}(s'_i, a'_i)$
- ❑ Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$
- ❑ Periodically update ϕ^- : $\phi^- \leftarrow \phi$

Improvements/extensions:

- ❑ Double DQN
- ❑ Prioritized experience replay
- ❑ Dueling architecture
- ❑ Q-learning with continuous actions

Recap: Summary of Q-Learning

□ Key idea:



- Full offline: fitted Q iteration (FQI)
- Online Q-learning with neural network approximations: DQN and variants
- Q-learning is **off-policy** and **model-free**
- Q-learning doesn't explicitly learn a **policy**
- Q-learning could be hard with continuous actions
- Important algorithmic ideas: replay buffer, target network, double Q learning, dueling, prioritized buffer, etc

Recap: DDPG

□ How to solve max or argmax with continuous actions?

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma Q_{\phi}(s', \operatorname{argmax}_{a'} Q_{\phi}(s', a')) \right)^2 \right]$$

□ Idea: **learn an approximate optimizer**

□ DDPG (deep deterministic policy gradient, Lillicrap et al, ICLR'16)

- Main idea: train another DNN $\mu_{\theta}(s)$ such that $\mu_{\theta}(s) \approx \operatorname{argmax}_a Q_{\phi}(s,a)$
- $\mu_{\theta}(s)$ is essentially a **policy**!
- How? Gradient ascent on $Q_{\phi}(s, \mu_{\theta}(s))$! Chain rule: $\frac{dQ}{d\theta} = \frac{dQ}{da} \frac{d\mu}{d\theta}$
- It is a deterministic actor-critic algorithm (more details in future lecture), but really can be viewed as an approximate Q learning with continuous action space

□ **Policy gradient** (today): can we directly optimize $\mu_{\theta}(s)$?

Outline

(Some black board derivation today!)

☐ Recap: Q-Learning

☒ Why and what is policy gradient

☐ Policy gradient theorem and REINFORCE

☐ Understanding and improving REINFORCE

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Why Policy Gradient

*Key philosophy: Why learn value function when all we need is a policy?
In some cases, policy might be easier to learn!*

□ Example: Linear Quadratic Regulator (LQR)

$$\begin{aligned} \min_{s,a} \mathbb{E}_w \left[\sum_{t=1}^T \frac{1}{2} s_t^\top Q s_t + \frac{1}{2} a_t^\top R a_t \right] \\ \text{s. t. } s_{t+1} = A s_t + B a_t + w_t, \forall t \in [1, T-1] \end{aligned}$$

- A standard setting in optimal control (more details in future lectures)
- Quadratic reward (no discounted factor): $-\frac{1}{2} s_t^\top Q s_t - \frac{1}{2} a_t^\top R a_t$
- Linear transition dynamics: $s_{t+1} = A s_t + B a_t + w_t$ (w_t is i.i.d. zero-mean noise)

Why Policy Gradient

*Key philosophy: Why learn value function when all we need is a policy?
In some cases, policy might be easier to learn!*

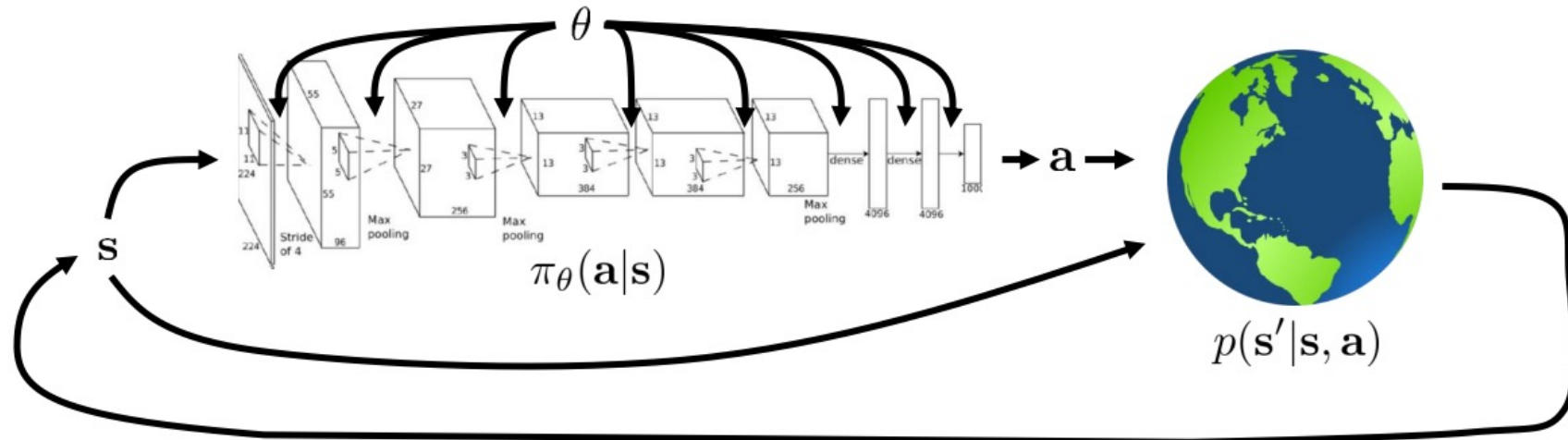
□ Example: Linear Quadratic Regulator (LQR)

$$\begin{aligned} \min_{s,a} \mathbb{E}_w \left[\sum_{t=1}^T \frac{1}{2} s_t^\top Q s_t + \frac{1}{2} a_t^\top R a_t \right] \\ \text{s.t. } s_{t+1} = A s_t + B a_t + w_t, \forall t \in [1, T-1] \end{aligned}$$

- When $T \rightarrow \infty$:
 - The optimal policy: $\pi(s) = -Ks$
 - The optimal value function: $V(s) = \frac{1}{2} s^\top P s$
- The optimal policy is linear, whereas the optimal value is quadratic!
- Learning $-Ks$ is often easier than learning $\frac{1}{2} s^\top P s$!

Policy Gradient (PG)

- ❑ Recap: the goal of RL
- ❑ The goal of policy gradient: directly optimize the policy π_θ via $\nabla_\theta J(\theta)$



$$\underbrace{p_\theta(s_1, \mathbf{a}_1, \dots, s_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(s_1) \prod_{t=1}^T \pi_\theta(\mathbf{a}_t | s_t) p(s_{t+1} | s_t, \mathbf{a}_t)$$

$J(\theta)$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, \mathbf{a}_t) \right]$$

Outline

(Some black board derivation today!)

- ☐ Recap: Q-Learning
- ☐ Why and what is policy gradient
- ☒ Policy gradient theorem and REINFORCE
- ☐ Understanding and improving REINFORCE

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

How to Evaluate $J(\theta)$?

$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

❑ Given some policy π_θ (might be very bad), how to evaluate its $J(\theta)$?

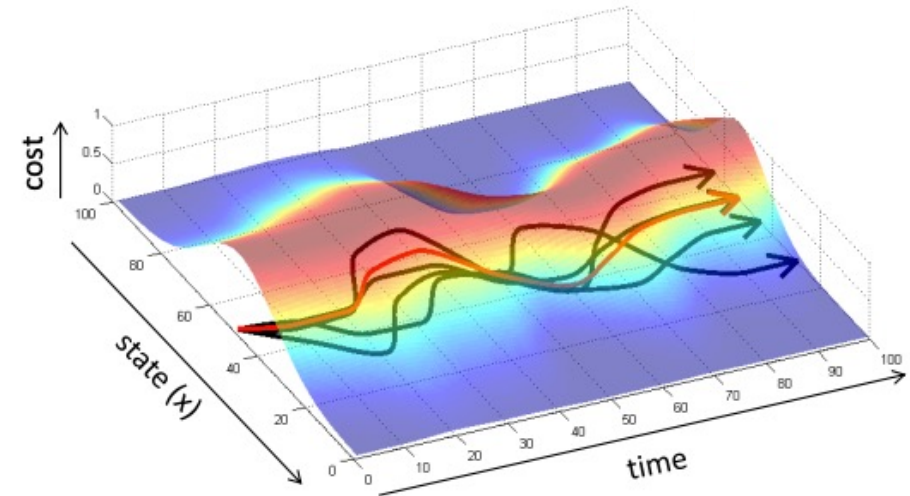
❑ Simple idea: Monte Carlo sampling!

- Sample N trajectories using π_θ
- Monte Carlo estimation of $J(\theta)$:

$$J(\theta) = E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

❑ **Policy gradient theorem** (next few slides):

- Can we estimate $\nabla_{\theta} J(\theta)$ using samples?



How to Compute $\nabla_{\theta} J(\theta)$?

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(\mathbf{s}_t, \mathbf{a}_t)}_{R(\tau)} \right]$$

- ❑ $R(\tau)$: total reward (return) of the trajectory τ
- ❑ $p_{\theta}(\tau)$: likelihood of the trajectory τ under policy π_{θ}
- ❑ $\nabla_{\theta} J(\theta)$ is hard to compute: **θ influences both the action selections and the distribution of states in which those selections are made**
- ❑ Policy gradient theorem:

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

- ❑ Why? See the magic on the black board!

How to Compute $\nabla_{\theta} J(\theta)$?

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(\mathbf{s}_t, \mathbf{a}_t)}_{R(\tau)} \right]$$

□ Step 1 (definition of expectation): $J(\theta) = E_{\tau \sim p_{\theta}(\tau)}[R(\tau)] = \int p_{\theta}(\tau) R(\tau) d\tau$

□ Step 2 (using the “useful identity”):

$$\nabla_{\theta} J(\theta) = \int \nabla_{\theta} p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = E_{\tau \sim p_{\theta}(\tau)}[\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

Useful Identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \nabla_{\theta} p_{\theta}(\tau)$$

How to Compute $\nabla_{\theta} J(\theta)$?

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{\sum_t r(\mathbf{s}_t, \mathbf{a}_t)}_{R(\tau)} \right]$$

□ Step 3 (simplify $E_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$):

$$\log p_{\theta}(\tau) = \underbrace{\log p(s_0)}_{\text{initial state, not a function of } \theta!} + \sum_t \log \pi_{\theta}(a_t | s_t) + \underbrace{\log p(s_{t+1} | s_t, a_t)}_{\text{transition dynamics, not a function of } \theta!}$$

□ Finally:

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)] = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

REINFORCE Algorithm: Monte Carlo PG

□ REINFORCE: estimate this expectation by Monte Carlo sampling!

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

□ Recall:

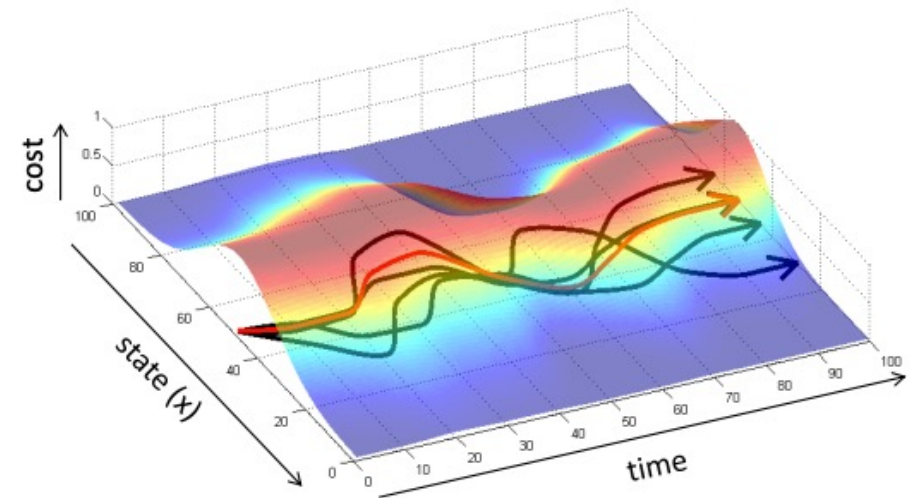
$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t})$$

□ Similarly ($\tau^i = s_{i,0}, a_{i,0}, s_{i,1}, a_{i,1}, \dots$):

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

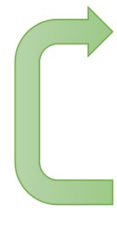
□ REINFORCE algorithm (Sutton et al., NeurIPS 1998):

- 1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
- 2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
- 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



REINFORCE Algorithm: Monte Carlo PG

□ REINFORCE algorithm (Sutton et al., NeurIPS 1998):

- 
1. sample $\{\tau^i\}$ from $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ (run the policy)
 2. $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(\mathbf{a}_t^i|\mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
 3. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

Policy Gradient Methods for Reinforcement Learning with Function Approximation

Richard S. Sutton, David McAllester, Satinder Singh, Yishay Mansour
AT&T Labs – Research, 180 Park Avenue, Florham Park, NJ 07932

Theorem 1 (Policy Gradient). For any MDP, in either the average-reward or start-state formulations,

$$\frac{\partial \rho}{\partial \theta} = \sum_s d^\pi(s) \sum_a \frac{\partial \pi(s, a)}{\partial \theta} Q^\pi(s, a). \quad (2)$$

Outline

(Some black board derivation today!)

- ☐ Recap: Q-Learning
- ☐ Why and what is policy gradient
- ☐ Policy gradient theorem and REINFORCE
- ☒ Understanding and improving REINFORCE

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Understanding REINFORCE (and PG)

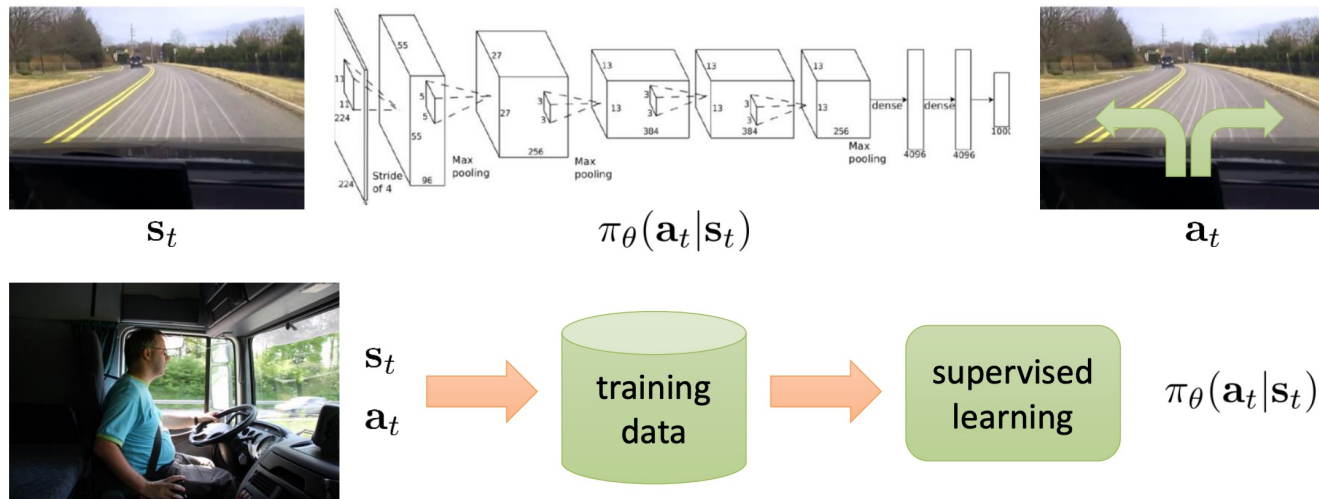
□ Why it makes sense?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

□ Recall what happens in imitation learning (BC, behavior cloning):

- Maximizing log likelihood of the policy π_{θ} on expert data!

$$\nabla_{\theta} J_{\text{BC}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$



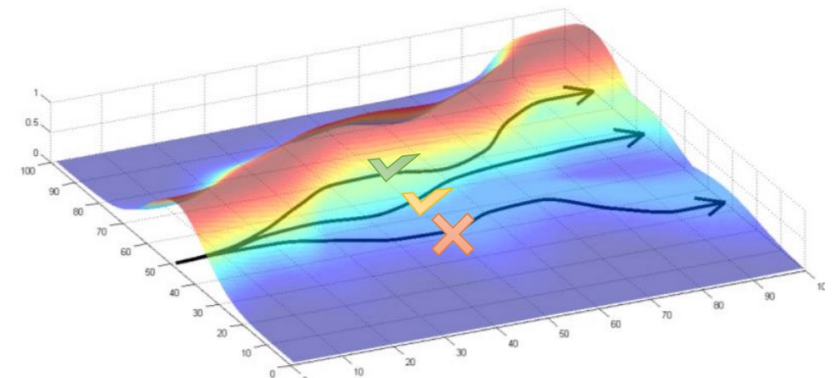
Understanding REINFORCE (and PG)

❑ Why it makes sense?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

- ❑ Without expert data: signal is from $R(\tau^i)$!
- ❑ Good stuff (bigger $R(\tau^i)$) is made more likely
- ❑ Bad stuff is made less likely
- ❑ Simply formalizes the notion of “trial and error”!

- 
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
 2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



Understanding REINFORCE (and PG)

❑ Why it makes sense?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

❑ Another way to explain it:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \frac{\nabla_{\theta} \pi_{\theta}(a_{i,t} | s_{i,t})}{\pi_{\theta}(a_{i,t} | s_{i,t})} \right)$$

❑ Gradient is roughly:

performance of an action sequence * the gradient of the probability of taking the action sequence / the probability of taking that action sequence

❑ Encourage of certain behavior if: (1) it gives high returns; (2) it appears rarely.

Understanding REINFORCE (and PG)

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

❑ Vanilla policy gradient (REINFORCE) is **on-policy**

❑ Why? $\tau \sim p_{\theta}(\tau)$ is from the current policy π_{θ} !

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

❑ Compare with **off-policy** Q-learning:

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s, a) - r(s, a) - \gamma \max_{a'} Q_{\phi^{-}}(s', a') \right)^2 \right]$$

Understanding REINFORCE (and PG)

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}_{J(\theta)} p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$
$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- ❑ Vanilla policy gradient (REINFORCE) is **on-policy**
- ❑ Why? $\tau \sim p_{\theta}(\tau)$ is from the current policy π_{θ} !

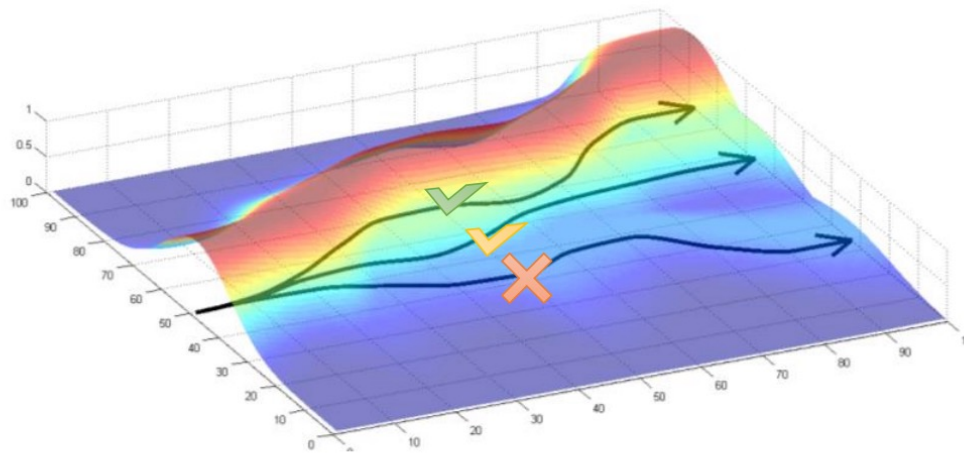
$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

- ❑ Policy gradient is **model-free**
- ❑ Why? It doesn't attempt to learn $p(s_{t+1} | s_t, a_t)$!

What is Wrong with REINFORCE?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N R(\tau^i) \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right)$$

- ❑ It has high variance! Need to collect long trajectories to evaluate $R(\tau^i)$
- ❑ Even worse: what if good samples have $R(\tau^i) = 0$?



Reducing Variance

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \right) \left(\sum_t r(s_{i,t}, a_{i,t}) \right)$$

□ Causality: policy at time t' cannot affect reward at $t < t'$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \underbrace{\left(\sum_{t'=t}^{\infty} r(s_{i,t'}, a_{i,t'}) \right)}_{\text{Reward to go!}}$$

Its expectation is $Q^{\pi}(s, a)$

□ Another form of **policy gradient theorem**:

- I.e., using $Q^{\pi}(s, a)$ to estimate return!

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

Reducing Variance: Baselines

□ Add a bias term does not affect gradient!

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

$$\begin{aligned} E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [b(s) \nabla_{\theta} \log \pi_{\theta}(a|s)] &= \int \int p_{\theta}(s) \pi_{\theta}(a|s) b(s) \frac{\nabla_{\theta} \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)} da ds \\ &= \int \int p_{\theta}(s) b(s) \nabla_{\theta} \pi_{\theta}(a|s) da ds \\ &= \int p_{\theta}(s) b(s) \int \nabla_{\theta} \pi_{\theta}(a|s) da ds \\ &= \int p_{\theta}(s) b(s) \nabla_{\theta} \int \pi_{\theta}(a|s) da ds \\ &= \int p_{\theta}(s) b(s) \nabla_{\theta} 1 ds = 0 \end{aligned}$$

□ Therefore

$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - b(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ The most common choice: $b(s) = V^{\pi}(s)$!

Thank You!